

Paradigmas II y Sistemas Operativos

Simulador de procesos y memoria de un Sistema Operativo

Cordeiro Mariano Angel - 44945214

<https://github.com/CordeiroM17/Proceso-CPU-Memoria>

5 de Octubre 2026

Introducción

El proyecto implementa, como librería TypeScript sin interfaz de consola, un simulador de un sistema operativo simplificado. Reproduce dos mecanismos centrales: la planificación de CPU Round-Robin con quantum configurable y la administración de memoria con particiones contiguas dinámicas, con tres políticas de asignación (First-Fit, Best-Fit, Worst-Fit), liberación y coalescencia de huecos.

Escenario simulado. Memoria principal de 1024 KB, inicialmente un único bloque libre. Los procesos ingresan al sistema, esperan memoria si no hay un hueco suficiente, pasan a la cola de Listos, se ejecutan de a un tick en la CPU y, al terminar, liberan su partición. El tiempo avanza con *avanzarTick()*.

Tabla 1

Clases del simulador y sus responsabilidades

Clase / Tipo	Responsabilidad
SimuladorSO	Motor del sistema. Mantiene colas (nuevos, esperando memoria, listos, bloqueados, terminados), el proceso en CPU, el reloj y las métricas de CPU. Orquesta cada tick y aplica Round-Robin.
Proceso	Modelo de PCB: pid, tamaño de memoria, tiempo de CPU total/restante, estado, quantum consumido, datos de E/S. Valida sus datos (esValido) y expone una vista (getVista).
AdministradorMemoria	Lista ordenada de particiones. Asigna con una estrategia, parte bloques, libera por pid, fusiona huecos adyacentes (coalescencia) y calcula métricas de memoria.
BloqueMemoria	Partición contigua: inicio, tamaño, libre/ocupada, pid propietario. Provee getFin y copiar.
IEstrategiaAsignacion	Contrato de una política: getNombre y seleccionar(bloques, requerido).
EstrategiaAsignacion (abstracta)	Template Method: seleccionar filtra los huecos libres donde el proceso entra y delega en elegir.

FirstFit / BestFit / WorstFit	Cada una implementa solo elegir: menor dirección, menor sobrante, mayor hueco. Ante empate se elige la dirección más baja.
AlgoritmoMemoria, EstadoProceso	Tipos unión de cadenas que restringen los valores válidos.
MetricasMemoria, VistaProceso	Interfaces de datos para métricas (RF09) y foto de solo lectura de un proceso (RF10).

Figura 1

Diagrama de clases del simulador

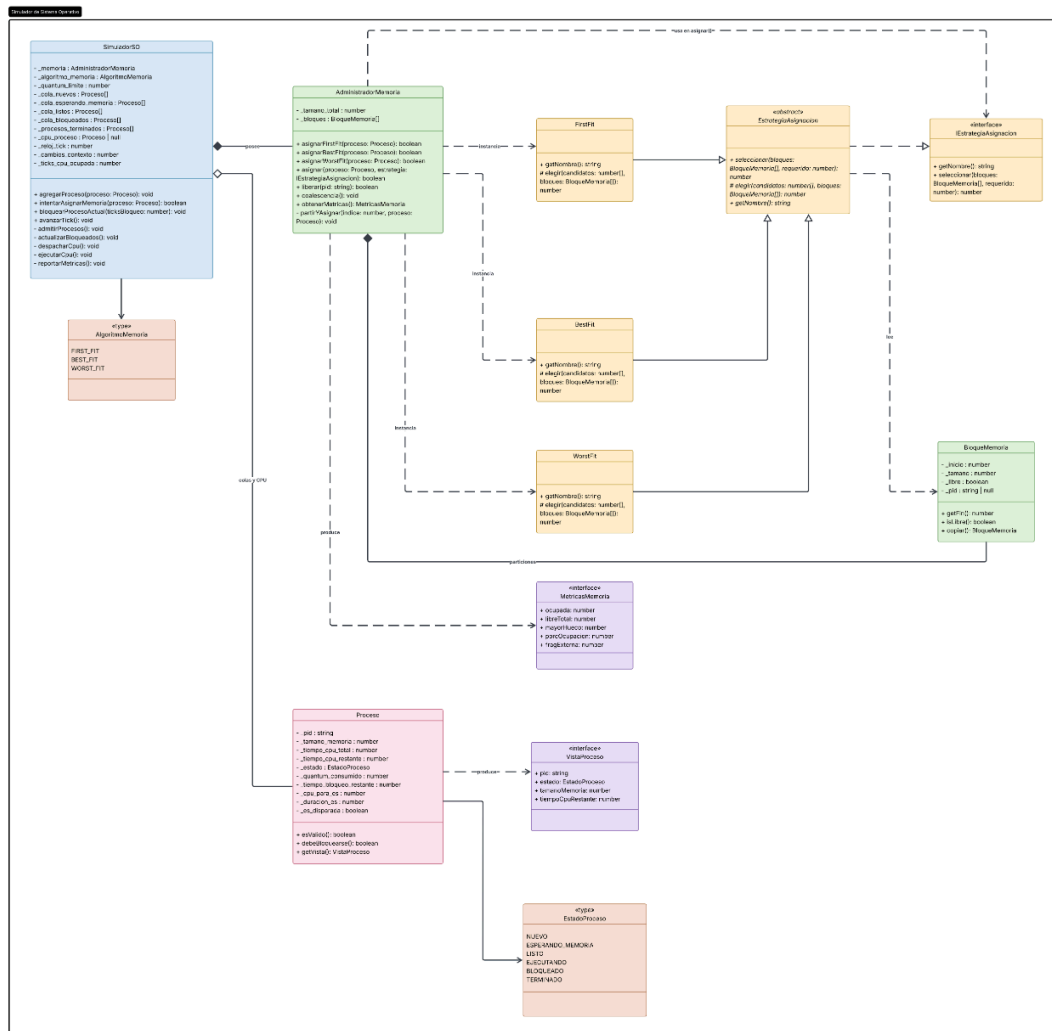


Figura 2

Diagrama de secuencia de admisión de un proceso y asignación de memoria (RF02, RF03, RF04)

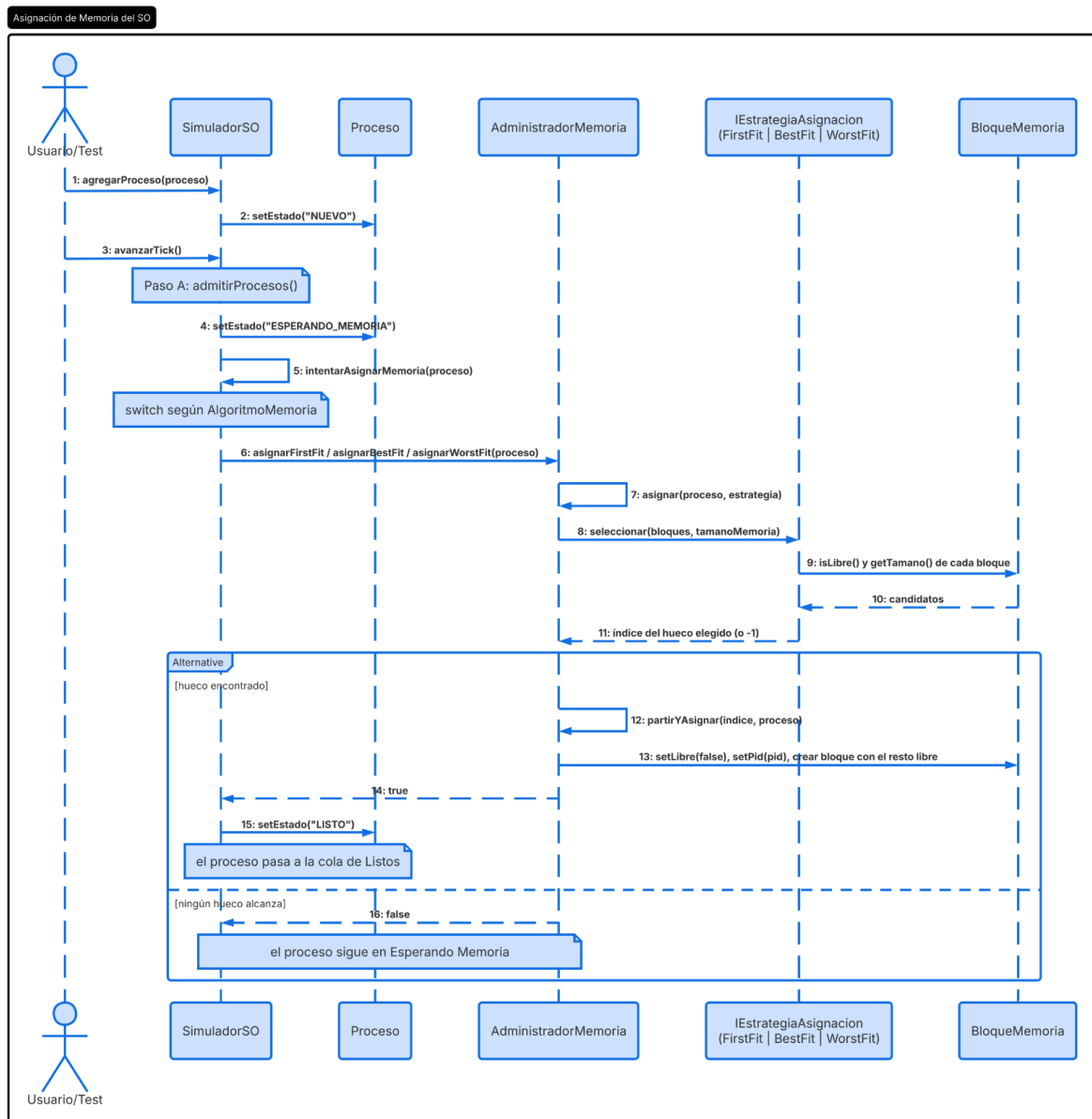


Figura 3

Diagrama de secuencia de un tick de simulación de Round-Robin (RF05, RF06, RF07, RF09)

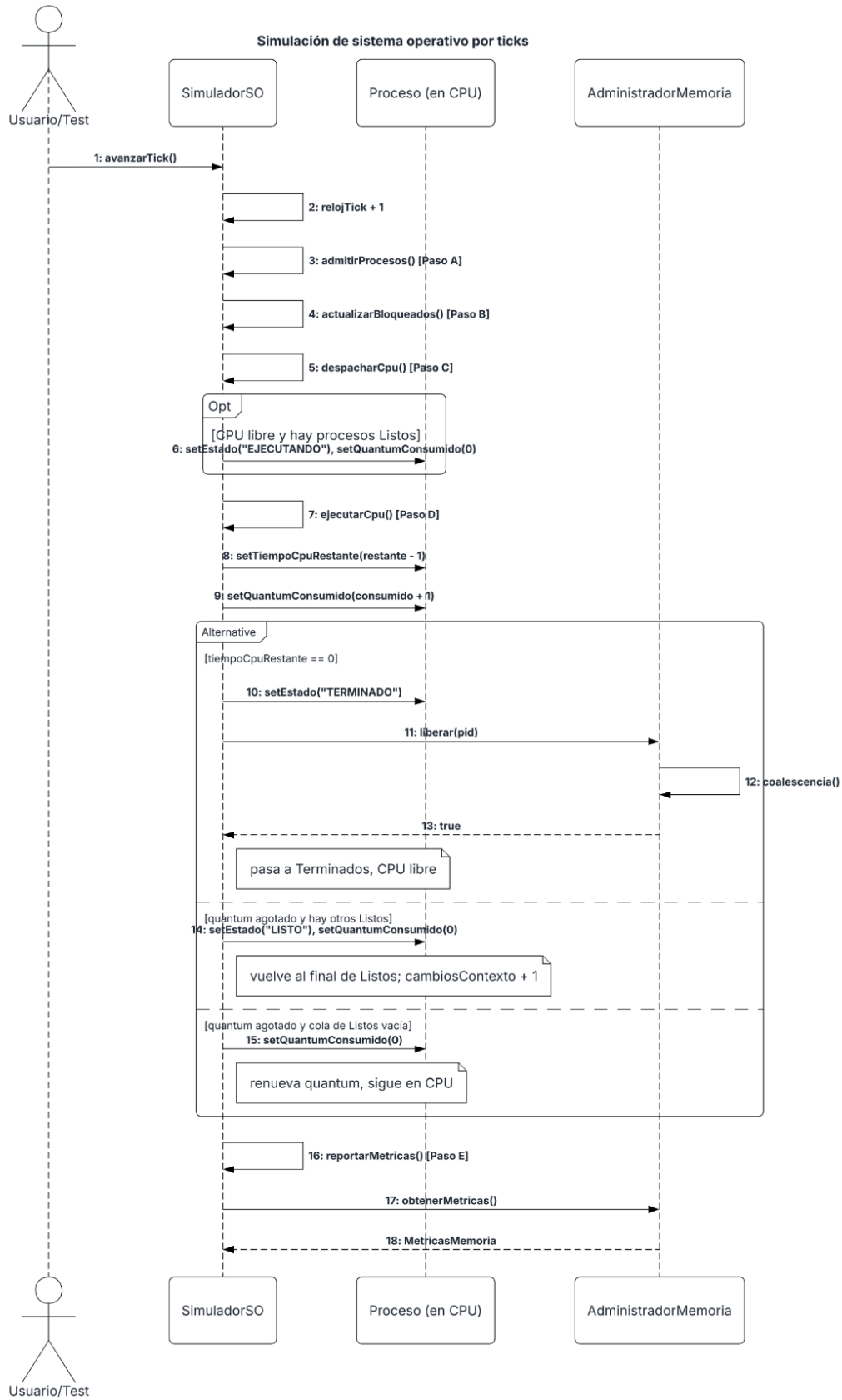
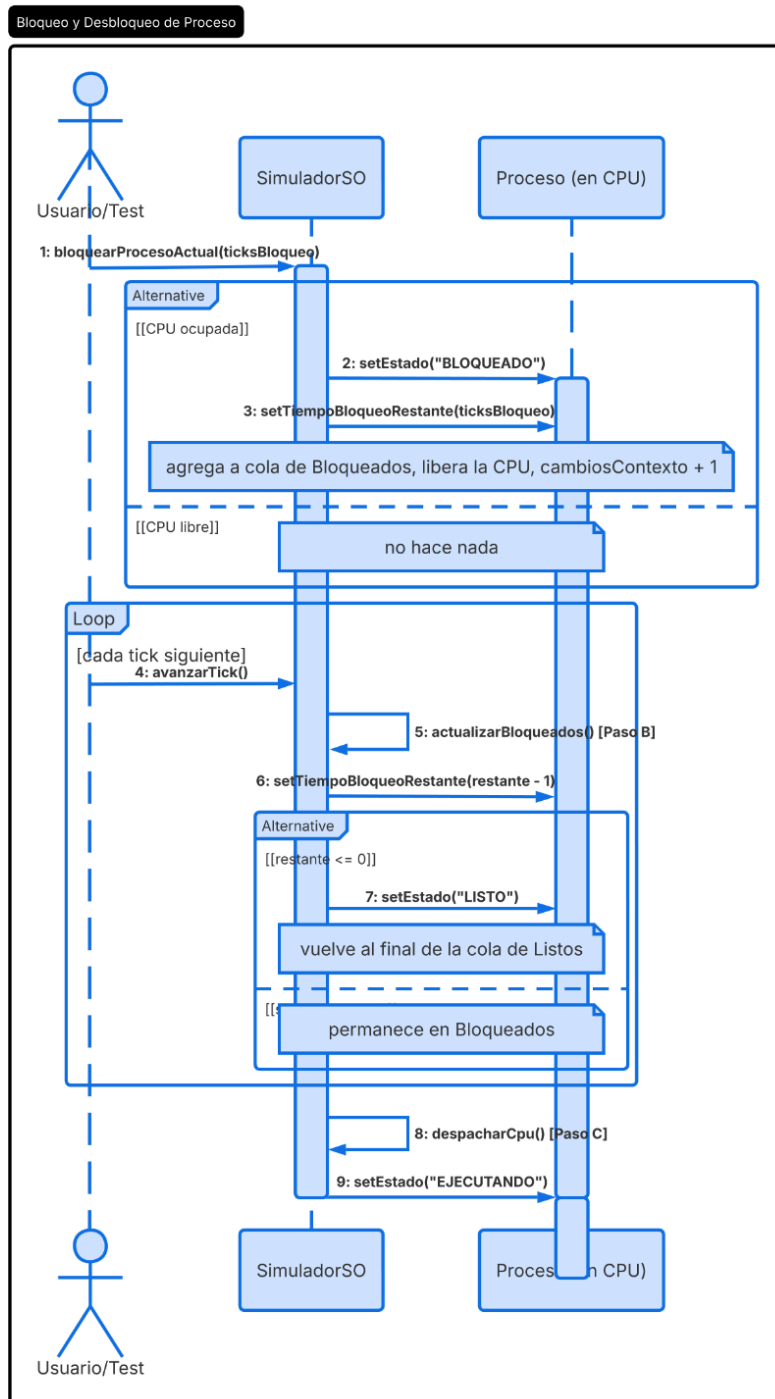


Figura 4

Diagrama de secuencia de bloqueo por E/S y retorno a Listos (RF08, parcial)



Decisiones de programación orientada a objetos

Encapsulamiento. Todos los atributos son privados (`_atributo`) con getters/setters. Los setters de datos que no deben cambiar (pid, tamaño, tiempo total, inicio de bloque) son protegidos. Las colas de SimuladorSO sólo se reasignan internamente.

Abstracción e interfaz. IEstrategiaAsignacion define el contrato de una política; el administrador depende de la abstracción y no de clases concretas.

Herencia y Template Method. EstrategiaAsignacion concentra el filtrado de candidatos (idéntico para las tres políticas). Cada subclase sobreescibe solo el criterio de elección.

Polimorfismo. asignar(proceso, estrategia) funciona igual con cualquier política. Los métodos asignarFirstFit/BestFit/WorstFit son delegaciones de una línea.

Resultados booleanos. Las operaciones que pueden fallar (asignar, liberar, intentar asignar memoria) devuelven boolean en lugar de lanzar excepciones: el flujo del simulador es más simple de leer y probar.

Tipos restringidos. EstadoProceso y AlgoritmoMemoria son uniones de literales: el compilado rechaza valores inválidos.

Política de memoria, estados y orden por tick

Con la asignación se buscan los bloques libres con tamaño $\geq$ requerido. **First-Fit** toma el de menor dirección; **Best-Fit** el de menor tamaño (menor desperdicio); **Worst-Fit** el de mayor tamaño. En empate gana la dirección más baja. El bloque elegido se parte: la parte ocupada por el proceso y, si sobra espacio, un nuevo bloque libre inmediatamente a continuación.

Liberación y coalescencia. liberar(pid) marca libre el bloque del proceso y ejecuta coalescencia(), que fusiona bloques libres adyacentes. Si el pid no existe devuelve false. Estados. NUEVO $\rightarrow$ ESPERANDO_MEMORIA $\rightarrow$ LISTO $\rightarrow$ EJECUTANDO $\rightarrow$ TERMINADO; desde EJECUTANDO puede volver a LISTO (fin de quantum) o pasar a BLOQUEADO y de ahí a LISTO.

Orden por tick. (1) se incrementa el reloj; (A) admitir procesos nuevos y reintentar asignar memoria a los que esperan; (B) descontar un tick a los bloqueados y devolver a Listos los que cumplieron; (C) si la CPU está libre, despachar el primero de Listos (FIFO); (D) ejecutar un tick: restar CPU, sumar quantum, y resolver fin de proceso / fin de quantum / renovación; (E) reportar métricas.

Cambios de contexto. Se cuentan cuando un proceso deja la CPU por quantum agotado habiendo otros Listos, o por bloqueo de E/S. La terminación de un proceso y la renovación de quantum sin competencia no se cuentan.

Requisitos

A continuación se presenta una tabla de los requerimientos funcionales pedidos por la consigna, que se hizo y cual es su estado final a la hora de la entrega.

Tabla 2*Tabla de requisitos funcionales*

RF	Estado	Clase / Metodo	Pruebas
RF 01 Configuración	Parcial	SimuladorSO.constructor (algoritmo, quantum); AdministradorMemoria(tamanoTotal)	SimuladorSO.test (quantum < 1 lanza error)
RF 02 Alta de procesos	Parcial	Proceso.esValido, SimuladorSO.agregarProceso	Proceso.test, SimuladorSO.test
RF 03 Asignación contigua	Cumple	AdministradorMemoria.asignar, partirYAsignar	AdministradorMemoria.test
RF 04 Políticas	Cumple	IEstrategiaAsignacion, EstrategiaAsignacion, FirstFit/BestFit/WorstFit	AdministradorMemoria.test, SimuladorSO.test (it.each)
RF 05 Liberación y coalescencia	Cumple	AdministradorMemoria.liberar, coalescencia	AdministradorMemoria.test
RF 06 Round-Robin	Cumple	SimuladorSO.ejecutarCpu, despacharCpu	SimuladorSO.test
RF 07 Orden por tick	Cumple	SimuladorSO.avanzarTick (fases A–E)	SimuladorSO.test (asignar antes de liberar)
RF 08 Bloqueo por E/S	No Cumple	Proceso.debeBloquearse existe, pero	SimuladorSO.test (bloqueo

		SimuladorSO no lo invoca; solo bloquearProcesoActual manual	manual)
RF 09 Métricas	Parcial	AdministradorMemoria.obtenerMetricas; SimuladorSO.cambiosContexto, ticksCpuOcupada	AdministradorMemoria.test, SimuladorSO.test
RF 10 Encapsulamiento	Parcial	Proceso.getVista, BloqueMemoria.copiar (no usados aún por el motor)	Proceso.test, BloqueMemoria.test

Resultados de coverage

Tabla 3

Tabla de resultados del coverage obtenido

Archivo	%Stmts	%Branch	%Funcs	%Lines
SimuladorSO.ts	99.23	96.29	100	99.21
Proceso.ts	97.87	100	95.83	97.87
BloqueMemoria.ts	94.44	100	90.9	94.44
WorstFit.ts	66.66	100	66.66	66.66
FirstFit.ts	50	100	50	50
Best.fit.ts	66.66	100	66.66	66.66
Total	97.81	98.64	95.09	97.74

Las líneas sin cubrir son los getNombre() de las tres estrategias y utilidades menores (copiar, getVista). La cobertura de líneas (97,87 %) supera el 90 % exigido.

Se define fragmentación externa = $(1 - \text{mayor hueco} / \text{memoria libre total}) \times 100$.
 Escenario ilustrativo sobre 1024 KB con tres huecos libres: 300 KB en 0, 200 KB en 400 y 324 KB en 700 (libre total 824 KB). Llega un proceso de 150 KB:

Política Hueco elegido Libre tras asignar Mayor hueco Fragmentación

- First-Fit 0 (300 KB) 674 KB 324 KB $\approx 51,93 \%$
- Best-Fit 400 (200 KB) 674 KB 324 KB $\approx 51,93 \%$
- Worst-Fit 700 (324 KB) 674 KB 300 KB $\approx 55,49 \%$

Worst-Fit deja la mayor fragmentación porque consume el hueco más grande y fragmenta lo que podría alojar procesos grandes futuros; First-Fit y Best-Fit empatan en este caso. Como caso mínimo, con huecos de 100 KB y 300 KB, un mayor hueco de 300 KB sobre 400 KB libres da 25 %.

La coalescencia reduce la fragmentación al fusionar huecos contiguos cuando un proceso termina, pero no puede eliminar la dispersión causada por procesos que permanecen entre huecos.

Limitaciones y trabajo pendiente

Tabla 4

Tabla de las limitaciones encontradas y el trabajo pendiente a realizar

Tema	Situación
Salida por consola	El código contiene console.log en SimuladorSO y imprimirMapa en AdministradorMemoria. La consigna pide una librería sin consola ni main; queda como mejora (reemplazar por eventos/métricas devueltas).
RF08	La E/S no se dispara automáticamente: debeBloquearse existe en Proceso pero el motor no la consulta.
RF01 / RF02	No hay memoria total configurable en SimuladorSO; agregarProceso no valida duplicados ni tamaño mayor que la RAM.
RF09	Uso de CPU y cambios de contexto se calculan pero solo se informan por consola;

	faltaría un método que los devuelva como objeto.
RF10	Los getters de colas devuelven las listas reales; getVista y copiar no son usados por el motor.
Diseño	No hay una clase Planificador separada: la lógica Round-Robin vive en SimuladorSO. Existe una única interfaz de dominio (IEstrategiaAsignacion).

Conclusiones

El simulador cumple el núcleo del problema: asignación contigua con tres políticas intercambiables por polimorfismo, liberación con coalescencia, planificación Round-Robin con orden de fases determinista y métricas de memoria. La separación entre la interfaz, la clase abstracta (Template Method) y las estrategias concretas permitió agregar una política sin tocar el administrador.

El trabajo permitió construir un simulador de un sistema operativo simplificado que integra dos mecanismos centrales de la gestión de recursos: la planificación de CPU Round-Robin y la administración de memoria con particiones contiguas dinámicas. El resultado cubre el núcleo del problema planteado: asignación con tres políticas intercambiables (First-Fit, Best-Fit y Worst-Fit), liberación con coalescencia de huecos, un orden determinista de fases dentro de cada tick y métricas de memoria. Las 46 pruebas automatizadas aprueban y la cobertura de líneas llega a 97,81 %, por encima del 90 % exigido (Tabla 5).